

第四章 区块链网络层

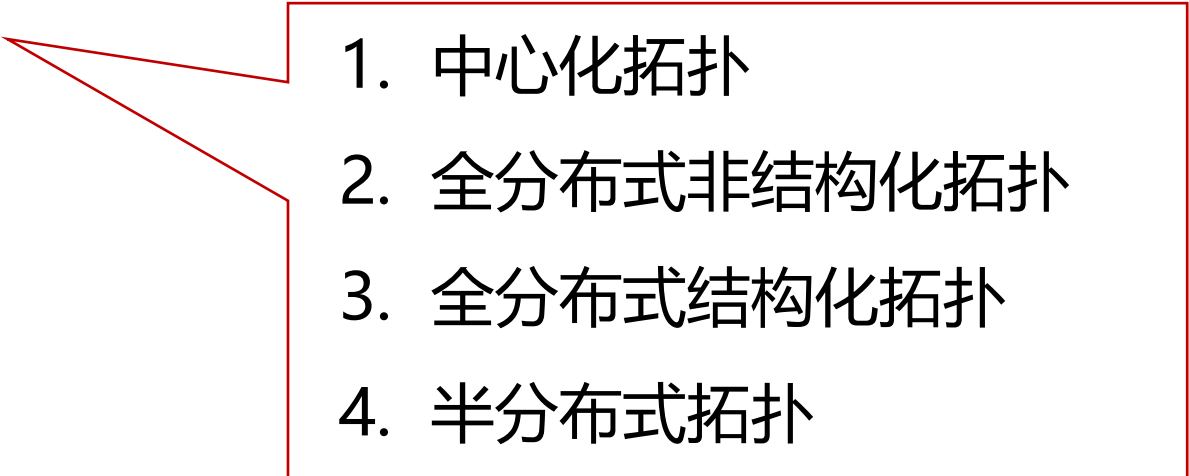
本章内容

4.1 P2P网络

4.2 比特币网络

4.3 以太坊网络

4.4 网络层安全

- 
1. 中心化拓扑
 2. 全分布式非结构化拓扑
 3. 全分布式结构化拓扑
 4. 半分布式拓扑

什么是P2P网络

➤ 不同的定义

- Intel：通过系统间的直接交换达成计算机资源与信息的共享
- IBM：由若干互联协作的计算机构成并具备如下特性之一：系统依存于边缘化设备的主动协作；每个成员同时扮演客户端和服务器的角色；系统应用的用户能意识到彼此的存在而构成一个虚拟或真实的群体

➤ 特点

- 节点彼此对等，既作为服务和资源的提供者，又作为服务和资源的获取者

区块链依靠P2P网络

➤ 可扩展性、健壮性

- P2P网络中的所有对等节点都可以提供带宽、存储空间以及计算能力等资源，随着更多节点的加入，系统整体的资源和服务能力也在同步地得到扩充

➤ 负载均衡

- P2P网络的资源分布在多个节点上，可以实现网络的负载均衡

➤ 去中心化

- 在区块链系统的P2P网络中，节点是信息的发送方和接收方，它们共同维护区块链

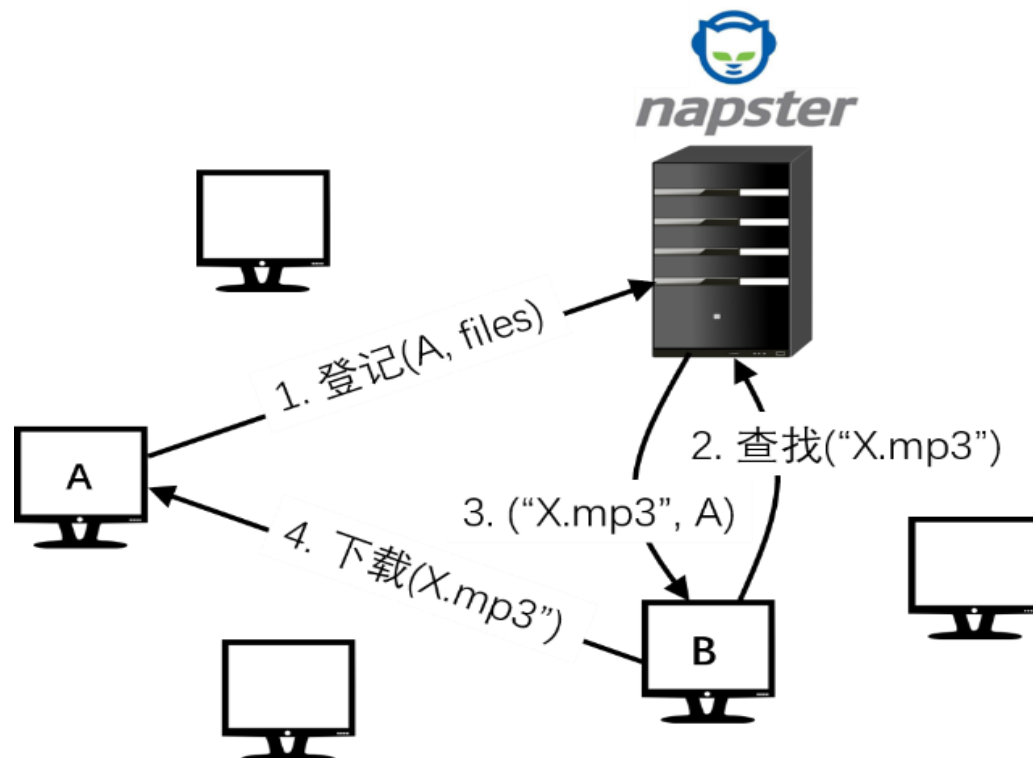
拓扑形式一

➤ 中心化拓扑

- 由一台中心索引服务器和多个客户端节点构成，并非纯粹的P2P网络
- 中心索引服务器用于保存接入节点的地址信息，向其他节点提供地址索引服务

➤ 特点

- 实现了文件查询和文件传输的分离，且维护简单
- 一旦中心索引服务器发生了故障，就会导致整个网络无法正常工作



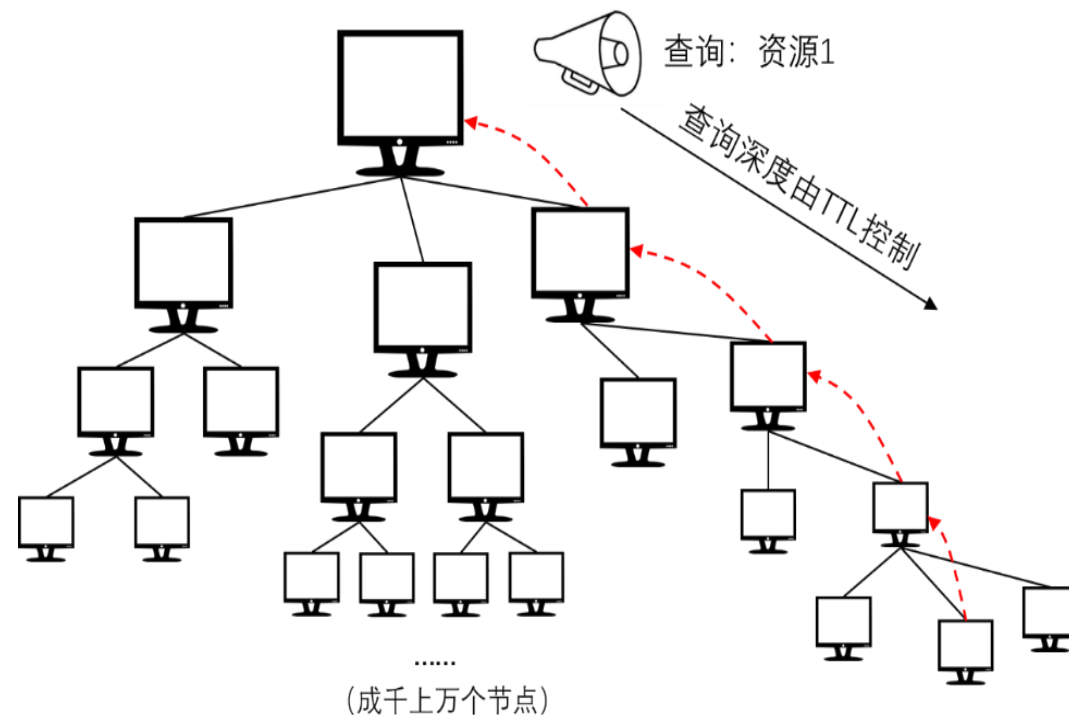
拓扑形式二

➤ 全分布式非结构化拓扑

- 没有使用中心索引服务器，其节点拥有真正的对等关系
- 洪泛（Flooding）数据广播，即节点会将接收到的消息向邻居节点转发，直到所有节点都接收到了这个消息或消息传播的深度到达一定的限制

➤ 特点

- 可能会出现广播风暴
- 实现快速的消息传播和资源查找



拓扑形式三

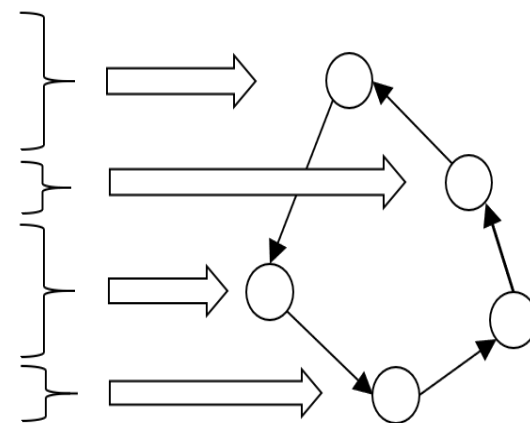
➤ 全分布式结构化拓扑

- 采用分布式散列表（Distributed Hash Tables, 简称DHT）来实现整个网络的寻址和存储，从而结构化地址管理
- 分布式散列表将存储着网络中所有资源信息的散列表划分成很多不连续的小块，分散地存储在多个节点上

➤ 特点

- 维护机制较为复杂
- 良好的健壮性、可扩展性和动态适应性

Key	Value
Fatemeh	Stockholm
Ali	California
Tallat	Islamabad
Cosmin	Bucharest
Seif	Stockholm
Amir	Tehran



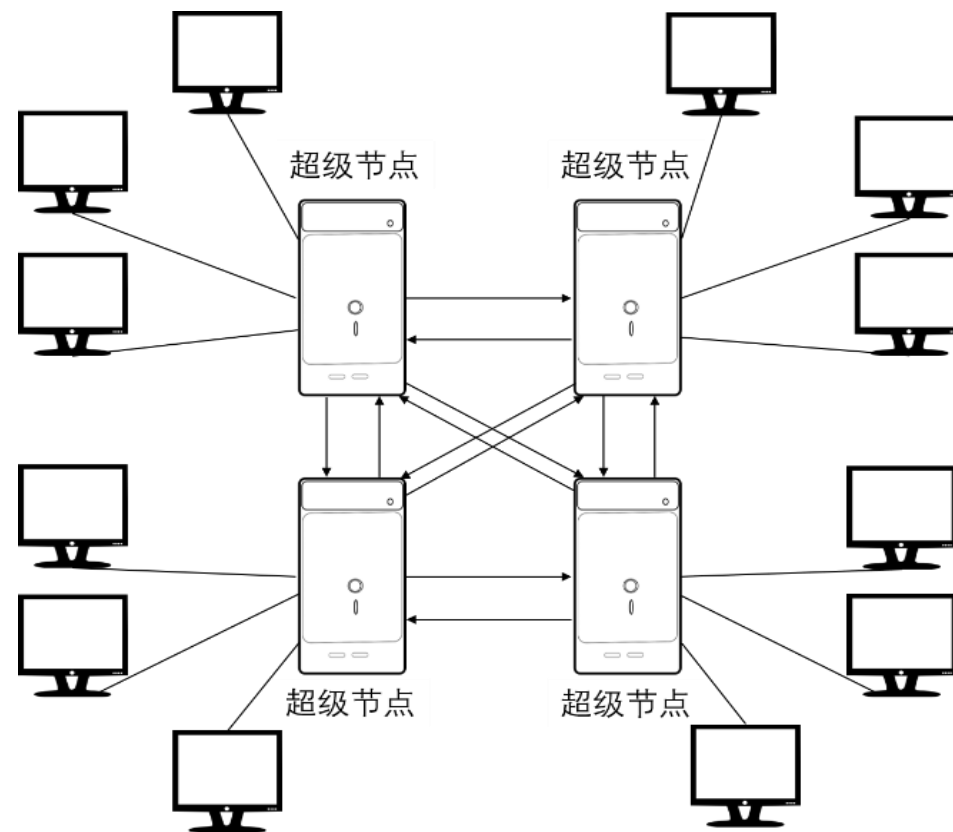
拓扑形式四

➤ 半分布式拓扑

- 将网络中性能较高的机器作为超级节点，每个超级节点存储着系统中其他部分节点的文件信息，执行维护这些节点的地址、文件索引等工作
- 超级节点之间形成一个高速的转发层，并与接入的普通节点形成一个自治的簇，簇内采用中心拓扑的P2P网络

➤ 特点

- 消除了网络拥塞的隐患，并在性能和可扩展性上具有一定的优势
- 对超级节点的依赖性较大



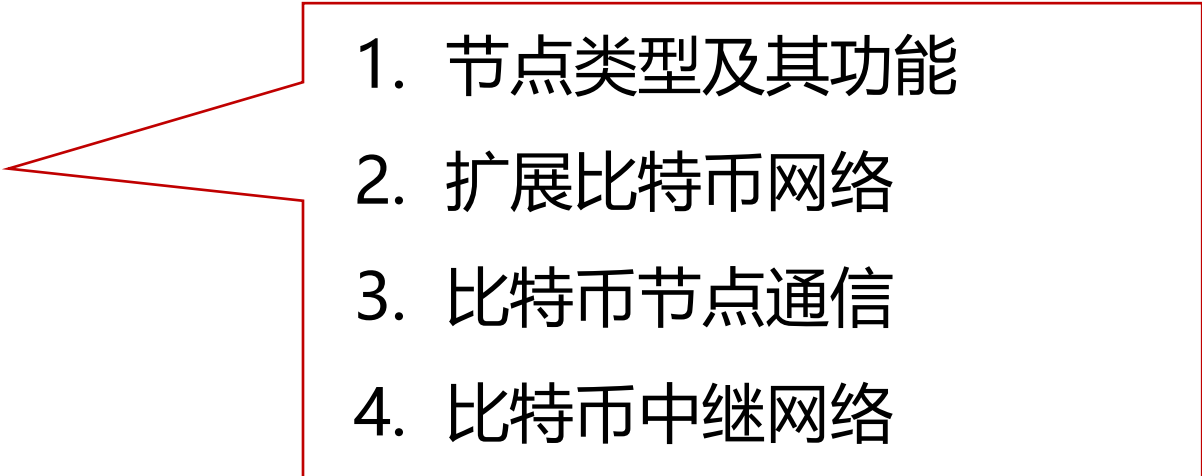
本章内容

4.1 P2P网络

4.2 比特币网络

4.3 以太坊网络

4.4 网络层安全

- 
1. 节点类型及其功能
 2. 扩展比特币网络
 3. 比特币节点通信
 4. 比特币中继网络

比特币网络

➤ 基于全分布式非结构化拓扑

- 网络中的节点彼此对等，不存在特权节点和索引服务器，通过共识机制使所有诚实的节点保存一致的区块链视图，从而实现去中心化控制

➤ 协议

- 比特币网络是依照比特币P2P协议运行的一系列节点的集合，其P2P协议建立在传输层的TCP协议之上，采用8333端口作为主网默认通信端口
- 比特币还运行着其他协议，如应用于矿池挖矿、轻量级或移动端比特币钱包中的Stratum协议。这些协议由网关路由服务器提供，通过比特币P2P协议接入到比特币网络，使得运行着扩展功能的网络节点连接到比特币主网络

节点类型

- 依照节点保存区块数据内容和是否能独立完成交易验证划分
- 全节点 (Full Node)
 - 拥有完整的区块链数据，可以独立地进行区块和交易的验证
- 轻节点 (Lightweight Node)
 - 只保存区块链数据的部分信息（如：区块头），不能独立地进行区块和交易的验证。它通过简易支付验证 (Simplified Payment Verification, 简称SPV) 方式向其他节点请求数据来完成支付验证

功能类型

- 依照节点承载的功能进行划分
- 钱包 (Wallet)
 - 可以支持比特币交易、查询等功能
- 矿工 (Miner)
 - 可以运行工作量证明算法来争夺创建新块的资格，从而赚取系统奖励的比特币以及交易手续费
- 完整区块数据存储 (Full Blockchain)
 - 存储着区块链的完整数据，可以独立地验证所有交易，不需要借助任何外来参考
- 网络路由 (Network Routing Node)
 - 连接着一定数量的节点，能帮助转发交易和区块数据，发现和维护节点间的连接

节点类型

- 依照节点的功能进行划分
- 核心客户端节点（Reference Client（Bitcoin Core））
 - 包含钱包、矿工、完整区块存储、网络路由四种功能
- 全节点（Full Block Chain Node）
 - 拥有完整的区块链数据，具有网络路由功能
- 独立矿工节点（Solo Miner）
 - 拥有完整区块链数据，具有路由功能和挖矿能力，能不依赖其他节点的算力单独进行挖矿
- 轻量级钱包（Lightweight（SPV）Wallet）
 - 包含钱包与路由转发功能

扩展比特币网络

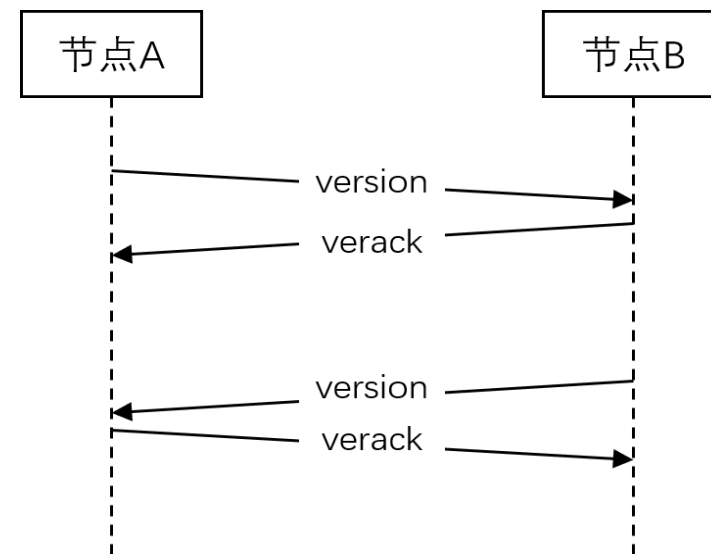
- 包含比特币P2P协议、矿池挖矿协议、Stratum协议（矿机与矿池软件之间的通信协议）及其他连接比特币系统组件的相关协议的整体网络结构被称为扩展比特币网络
- 常见的扩展节点有
 - 矿池协议服务器（Pool Protocol Server）：常作为比特币网络与其他矿池挖矿节点的网关路由
 - 挖矿节点（Mining Node）：一种轻量级节点，包含挖矿功能，但不包含区块链数据，必须依赖矿池服务器维护的全节点进行工作，通常运行Stratum协议或其他矿池挖矿协议
 - 轻型Stratum协议钱包（Lightweight (SPV) Stratum Wallet）：运行在Stratum协议下包含钱包功能的节点

比特币节点通信

- 为了能够参与协同运作，节点需要发现网络中的其他节点并与它们建立通信连接
- 新加入的节点通常采用下面两种方式进行节点发现
 - 使用“DNS种子”（DNS seeds）来查询DNS。比特币客户端会维护一个记录长期稳定运行节点的列表，这些节点也被称为种子节点，种子节点能提供比特币节点的IP地址列表。通过与种子节点进行连接，新节点可以快速发现网络中的其他节点
 - 通过-seednode命令指定一个比特币节点的IP地址作为比特币种子节点。节点会和这个种子节点进行连接以发现新节点

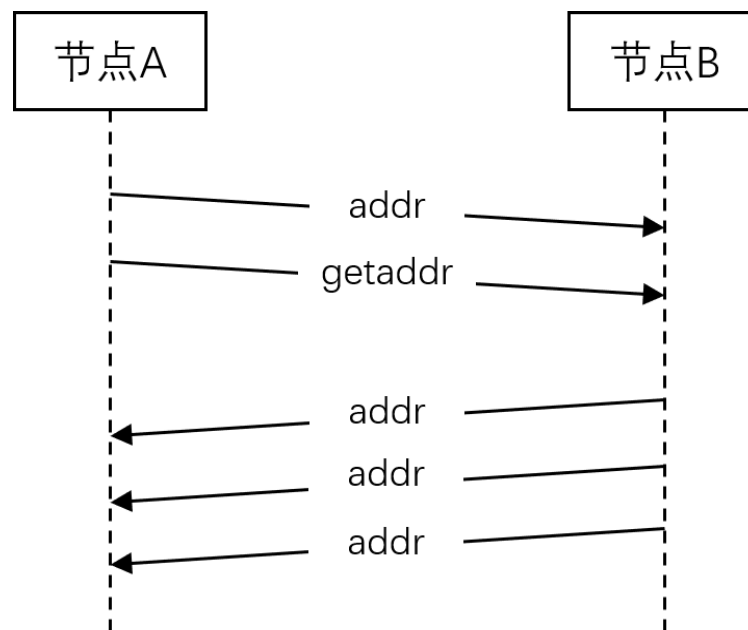
节点握手

- 节点发现之后，节点间进行“握手”连接实现信息的交换
- 节点A先向节点B发送version信息，包括以下内容
 - nVersion: 客户端采用的比特币P2P协议版本
 - nLocalServices: 一组该节点支持的本地服务列表，当前仅支持NODE_NETWORK
 - nTime: 当前时间
 - addrYou: 当前节点可见的远程节点的IP地址（节点B的IP）
 - addrMe: 当前节点的IP地址（节点A的IP）
 - subver: 指示当前节点运行的软件类型的子版本号
 - BestHeight: 当前节点区块链的区块高度（初始为0，即只包含创世区块）



地址传播和发现

- 当节点A和节点B的握手连接建立之后，节点B可以转发节点A的地址，让新节点被更多节点接收，并进一步向节点A提供节点引荐
- 节点A将一条包含自己IP地址的addr消息发送给节点B，节点B会将这条addr消息转发给节点B的相邻节点，使得节点A的信息会在网络中广播出去，被更多节点发现
- 此外，节点A可以向节点B发送getaddr请求，要求节点B向其发送已知的其他节点的IP地址。通过这种方式，可以实现比特币地址的传播和发现

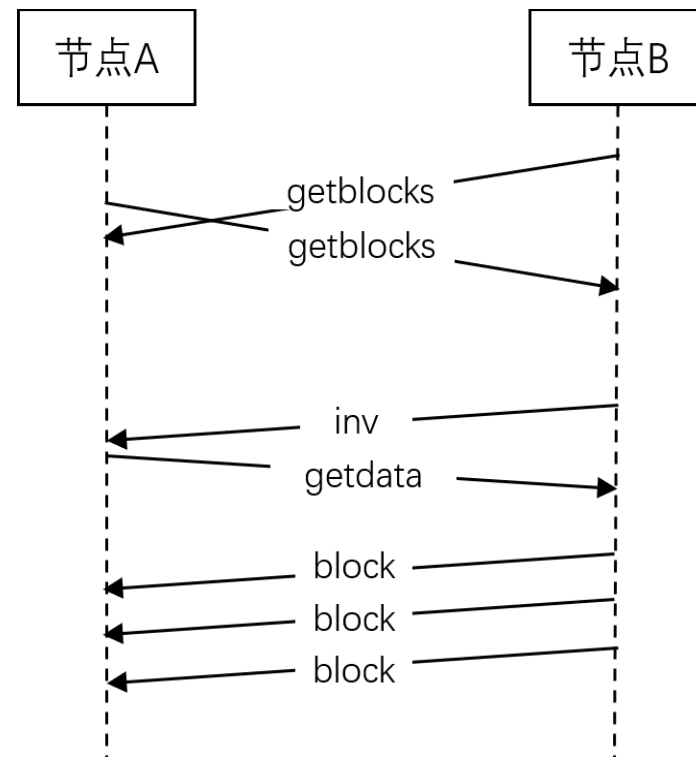


地址管理

- 比特币节点使用tried和new列表管理网络中节点的IP地址
 - tried列表包含了64个桶 (bucket) , 每个桶可以存储这个节点成功建立过入连接或出连接的64个不同的地址。tried列表还保存最近一次成功连接到该节点的时间戳。
 - 当插入节点信息时遇到桶已满的情况, 如果节点地址已存在于桶中, 可以直接更新时间戳; 否则, 会从桶中随机选出四个节点, 其中最远一次与节点成功建立连接的节点将被替换为新节点的信息, 并被插入到new列表中。
 - new列表包含了256个桶, 每个桶可以为节点保存64个尚未成功发起连接的节点地址。节点会利用从种子节点或从addr消息中学习的信息来填充new列表。
 - 如果遇到桶已满的情况, 会遍历桶中的所有节点, 将时间戳超过30天或多次尝试连接不成功的节点移出桶。

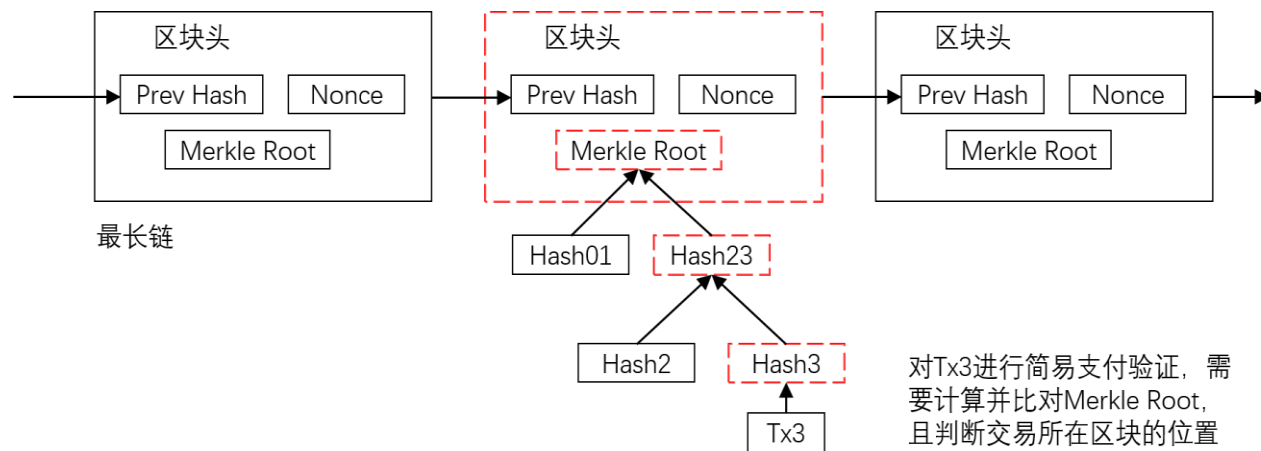
全节点区块同步

- 一个全节点在连接到其他节点以后，需要构建完整的区块链数据
- 节点A先向节点B发送version信息，包括以下内容
 - 在初始握手的version消息中，由于包含了BestHeight字段，节点可以了解对等节点的区块高度。
 - 然后节点间交换getblocks消息，其中包含本地区块链顶端区块的哈希值，通过判断接收到的哈希值是否属于顶端区块，节点可以判断哪一方拥有较多的区块信息。
 - 接着，拥有较多区块信息的一方会识别出其他节点缺少的区块数据，通过inv消息（每一个inv消息只包含500个区块的Hash信息，限制每次同步区块数量可以减少新节点同步区块对网络造成的影响）分批将对等方缺少的区块的哈希值传播出去。
 - 拥有较少区块的一方会向所有已连接的节点发送getdata信息继续请求全区块数据，并根据inv消息的哈希值找到自己缺少的区块数据进行读取。



SPV节点通信

- 一个全节点在连接到其他节点以后，需要构建完整的区块链数据
- 借助Merkle树算法，可以通过以下步骤进行简易支付验证
 - 进行区块头同步
 - 寻找包含该交易哈希的区块，验证区块头是否包含在最长链中
 - 获取构造Merkle树所需的交易哈希，计算Merkle树根的哈希
 - 若计算出的哈希与区块中的Merkle树根的哈希值相等，则交易存在于区块中
 - 根据区块头所处的位置，判断交易得到的确认数。如果交易已经经过六次确认，则完成对交易的验证



加密和认证连接

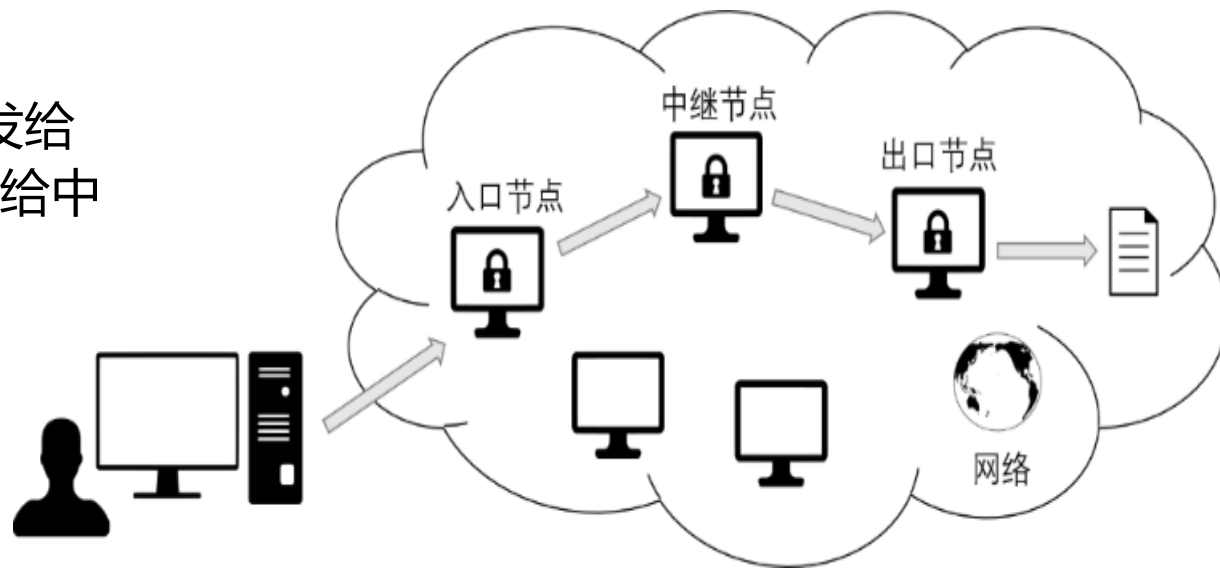
➤ 增加比特币网络的隐私性和安全性的方法

➤ Tor网络

- Tor软件将web请求进行多层加密，随机发给一个入口节点，入口节点第一次解密后发给中继节点解密
- 经过多次中继节点解密，最后到出口节点
- 出口节点最后一次解密后发给目的地址

➤ 特点

- 只有入口节点知道发送地址，出口节点知道目的地址
- 每一跳都有独立密钥



加密和认证连接

- 增加比特币网络的隐私性和安全性的方法
- P2P对等认证和通信加密
 - 请求方发送AUTHCHALLENGE（含对等体公钥哈希）
 - 响应方验证成功后发送AUTHREPLY（含签名）
 - 请求方验证成功后发送AUTHPROPOSE（含客户端认证公钥哈希）
 - 响应方验证成功后发送AUTHCHALLENGE（含客户端公钥哈希）
 - 请求方验证成功后发送AUTHREPLY（含签名）
 - 响应方验证成功后授予访问权限

交易池

➤ 交易池 (Transaction Pool)

- 每个节点会在本地维护一个记录已被网络发现但未被区块链所记录的交易的临时列表

➤ 孤立交易池 (Orphan Pool)

- 父交易未被节点所知的且未被记录在区块链上的交易，节点会将其存入孤立交易池

➤ UTXO池 (UTXO Pool)

- 包含着所有UTXO的集合，且只包含已确认交易

比特币中继网络

- 尝试最小化矿工之间传输块的延迟的网络
- FIBRE
 - 基于UDP的中继网络，它通过发送额外的数据来补偿数据包丢失，并基于Bitcoin core客户端中的压缩块进一步减少传输的数据量和网络延迟
- Falcon
 - 通过传播块的部分来减少网络延迟，使得节点不用一直等待直到接收到完整的块

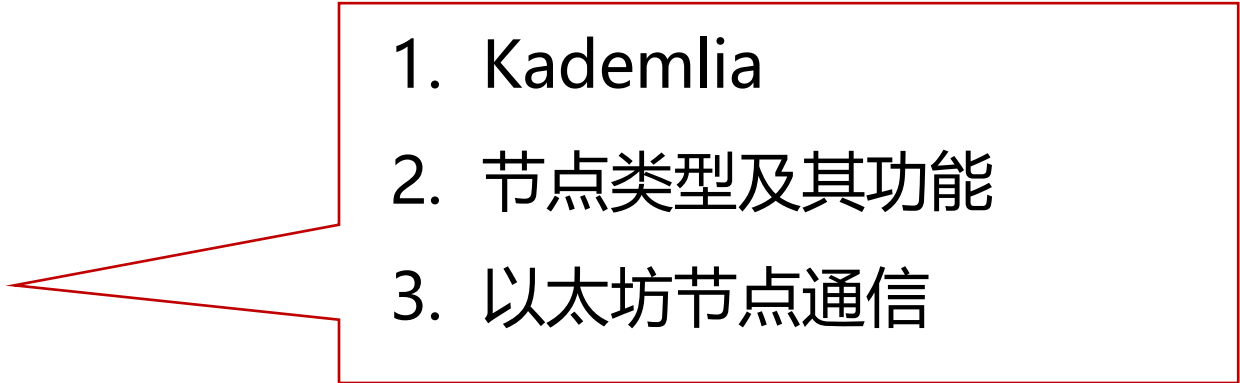
本章内容

4.1 P2P网络

4.2 比特币网络

4.3 以太坊网络

4.4 网络层安全

- 
1. Kademlia
 2. 节点类型及其功能
 3. 以太坊节点通信

以太坊网络

➤ 基于全分布式结构化拓扑

- 基于分布式散列表技术，可以在分布式环境下进行快速路由和数据定位

➤ 协议

- 以太坊的P2P网络是一个完全加密的网络，提供了UDP（主网推荐端口为30301）和TCP（主网默认端口为30303）两种连接方式，其中UDP主要用于P2P节点发现，TCP主要用于数据传输与交互

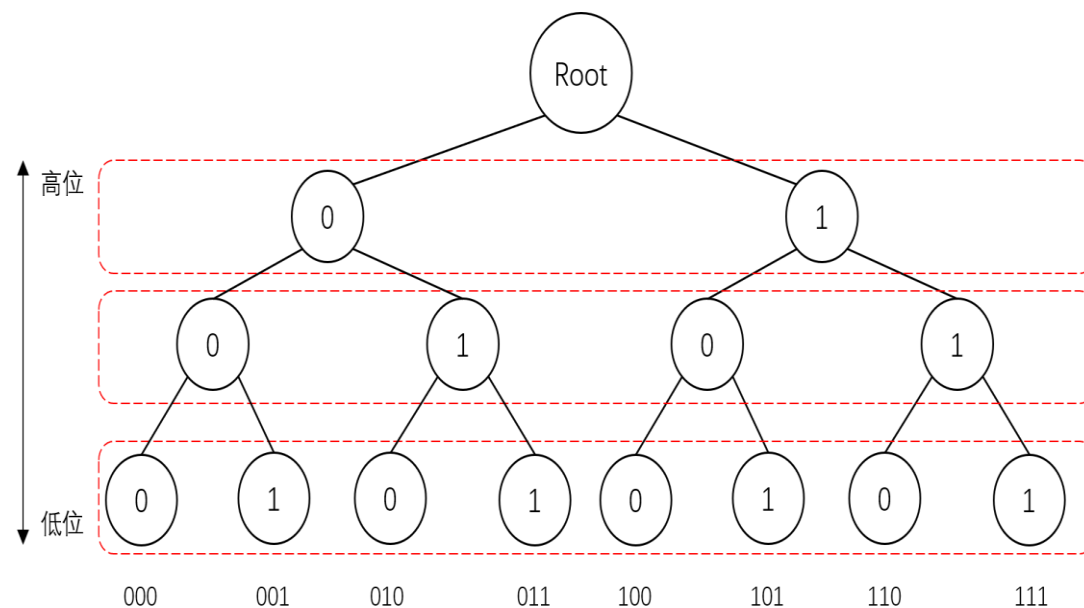
Kademlia

➤ Kad

- 一种分布式散列表技术，能将所有的信息作为哈希表的条目进行存储，分散地由各个节点来维护，从而以全网的方式构成巨大的分布式散列表，实现快速路由和准确定位

➤ 所有节点被表示为二叉树的叶子节点

- 每一个节点都被分配一个随机生成的160bit的节点ID作为标识符



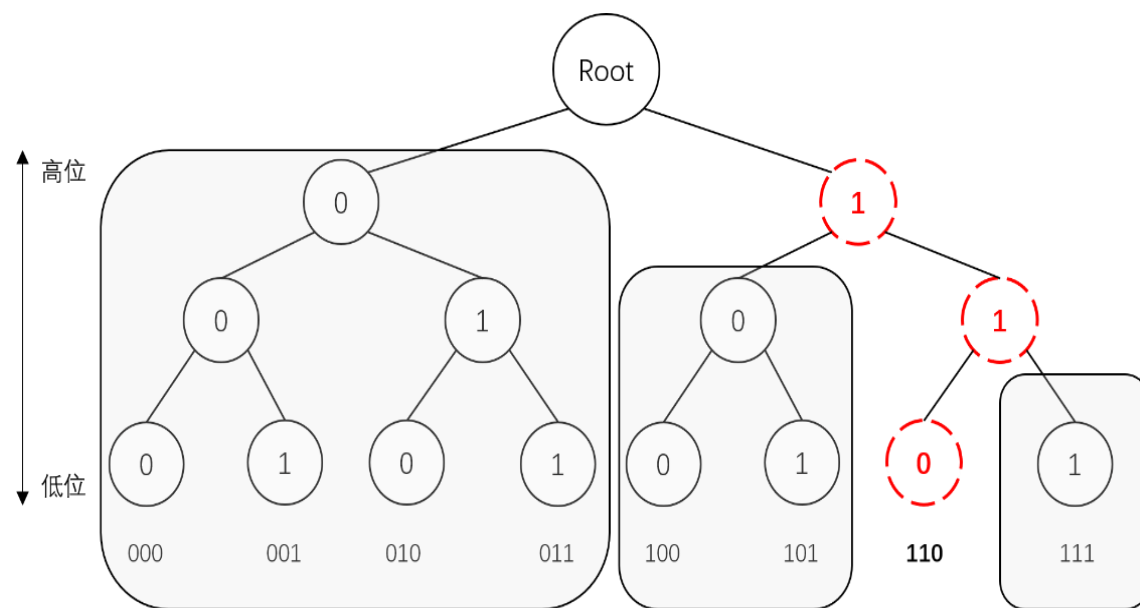
网络拆分

➤ 节点距离

- 假设ID空间是3bits, 那么ID为001和100的节点的距离为:
- $d(1,4)=d(001,100)=001 \oplus 100 = 101 = 5$

➤ 网络拆分

- 按ID经过的节点拆分子树
- 通过递归路由, 每次得到离目标节点最近的节点, 从而不断逼近并最终到达目标节点



K桶

➤ Kbucket

- 节点ID空间为n bit，每个节点从自己的视角拆分完子树后，可以得到最多n个子树
- 每个节点需要记录每个子树的k个节点，记录一棵子树中k个节点的列表被称为一个K桶

➤ K桶实际上就是路由表

- 每一个K桶i，它会存储和节点距离在 $[2^i, 2^{(i+1)})$ 范围内的k个节点的状态信息
- 每个K桶内部信息存放的位置根据上次访问的时间顺序排列，最早访问的节点会放在头部，最新访问的节点会放在尾部
- 以太坊网络对节点ID（这里指使用Eckey算法生成的512bit的网络节点公钥）使用sha3算法生成一个256bit的哈希码，每个节点有256个K桶，其中K桶的k值是16

K桶[0]: 距离[1,2)	Node 0	Node 1		Node k
K桶[1]: 距离[2,4)	Node 0	Node 1		Node k
K桶[2]: 距离[4,8)	Node 0	Node 1		Node k
.....				
K桶[n]: 距离 $[2^n, 2^{n+1})$	Node 0	Node 1		Node k

最近最少访问

最新访问

节点类型

- 依照分片后的功能划分
- 超级全节点 (Super-full node)
 - 下载了全部主链数据和与主链中每个分片有关的数据，旨在充分验证一切，可以在单个分片遭到破坏或攻击的情况下保持系统的可用性
- 顶级节点 (Top-level node)
 - 只处理所有主链区块，包括分片中的区块头和签名，不下载分片区块上的全部数据
- 单分片节点 (Single-shard node)
 - 充当顶级节点的角色，同时会完全下载和验证它关注的特定分片上的每一个校验规则
- 轻节点 (Light node)
 - 只下载和验证主链区块的区块头数据

节点通信

➤ Kad节点加入

- 获取任意一个已加入Kad网络中的节点的信息，将其加入对应的K桶中，并向该节点针对自己的ID发起查询请求
- 对等节点收到请求后，会按照与新节点的距离对自己的K桶进行更新，然后返回k个与新节点最近的节点
- 新节点收到返回的节点信息后，将这些节点加入自己的K桶，并向这些节点发送查询请求，如此往复，从而建立自己的路由表

➤ 节点退出

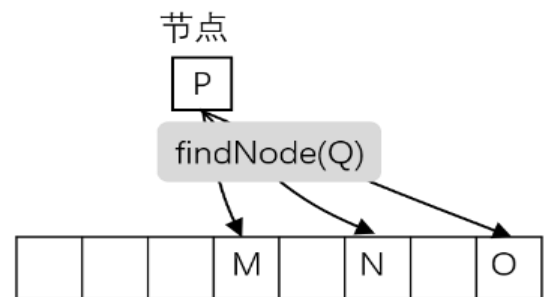
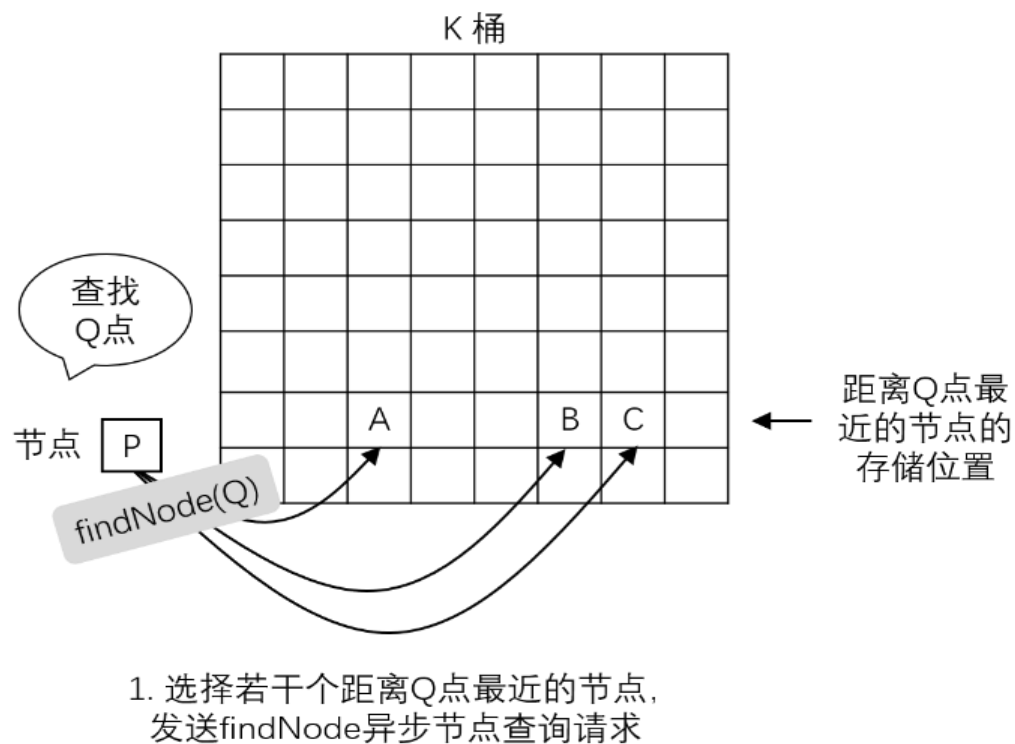
- 节点在维护自己的K桶时会主动把没响应的节点从自己的K桶中移除

K桶维护

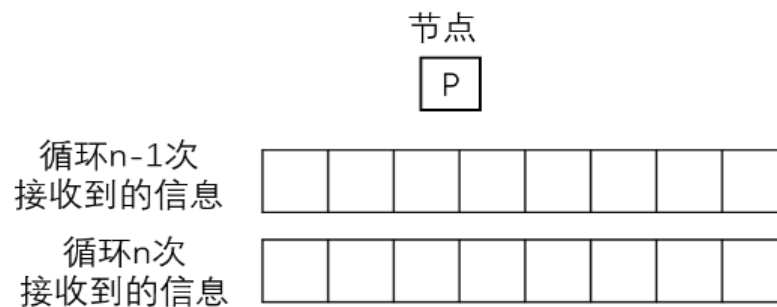
- 更新时基本遵循抛弃最近最少访问节点的原则
 - 计算节点本身与新节点的距离，根据这个距离，选择对应的K桶进行操作
 - 如果新节点的信息在这个K桶中，则将其信息移动到列表尾部
 - 如果新节点的信息不在K桶中，则分为两种情况：
 - 在K桶未满的情况下，直接将该信息添加到列表尾部；
 - 在K桶已满的情况下，先在列表头部检查最早访问的节点是否有响应。如果有响应，则将头部节点移动到列表尾部，并忽略新节点信息；如果没有响应，则将头部节点信息抛弃，并将新节点信息添加到列表尾部。保留在线时间长的节点的信息是因为在线时间长的节点在下一个时间段继续保持在线的概率可能会更大

Kad网络节点查询

- 定义：已知某个节点的ID，查找当前节点与目标节点距离最短的k个节点所对应的网络信息（不能保证被查找的节点一定存在在网络上）



2. 节点 P 接收查询返回信息，更新 K 桶，并向没有请求过的节点发送查询请求



3. 重复查询步骤直到前后两次查询结果相等

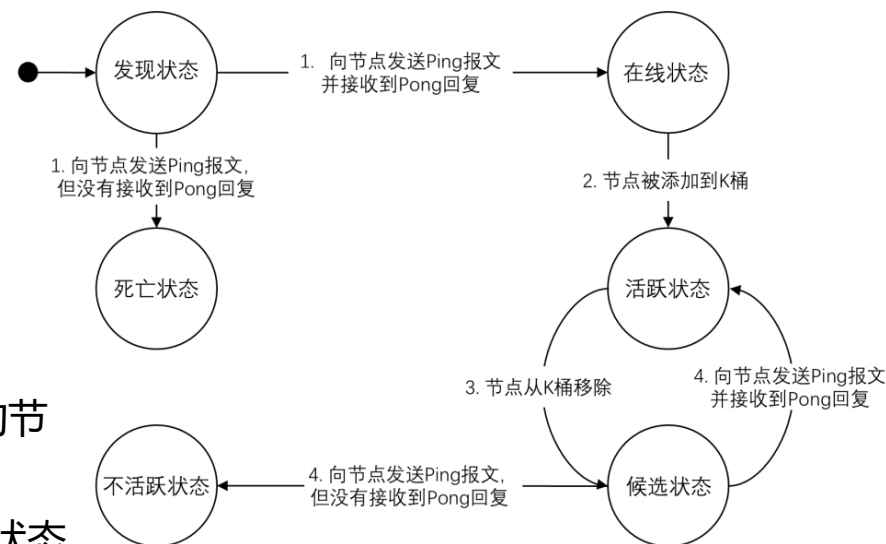
网络节点发现

➤ 四种报文命令

- 用于探测对等节点是否在线的Ping命令
- 用于响应Ping报文的应答命令Pong
- 用于向对等节点请求查询邻居节点的FindNode命令
- 用于回传找到的邻居节点列表的Neighbors命令

➤ 六种生命周期

- 发现状态 (discovered)：引导节点、从持久化文件加载的节点、被引荐的节点、接收到Ping报文的节点
- 在线状态 (alive)：在节点发现状态回复Pong报文后，节点会被置为在线状态
- 活跃状态 (active)：节点处于K桶时的状态
- 候选状态 (evictcandidate)：K桶满时，活跃状态的节点被新节点替代后会暂时被置为此状态
- 不活跃状态 (noactive)：如果节点向处于候选状态的节点发送Ping报文，且该候选状态的节点长时间没有响应，那么候选状态的节点会被置为不活跃状态
- 死亡状态 (dead)：节点在规定时间内没有返回Pong报文会进入的状态，这是最终状态



加密和认证连接

➤ 阶段一：密钥交换

- 节点A生成随机密钥对和随机数，用自己的私钥对节点B的公钥执行ECDH算法，将结果和随机数异或后使用随机生成的私钥对其签名，然后将随机数、签名、节点A的公钥、版本号打包后用节点B的公钥进行加密作为请求认证的握手报文进行发送
- 节点B使用自己的私钥对收到的握手报文进行解密，得到节点A生成的随机数、签名、公钥和版本号。
- 节点B生成随机密钥对和随机数，用自己的私钥和节点A的公钥执行ECDH算法，根据得到的结果和签名推导出节点A的随机公钥，利用随机私钥和节点A的随机公钥生成共享密钥，然后把自己的临时公钥和随机数用发起者的公钥进行加密并发送
- 节点A对节点B的消息进行解密获得节点B的临时公钥和随机数，利用ECDH算法计算出共享密钥

加密和认证连接

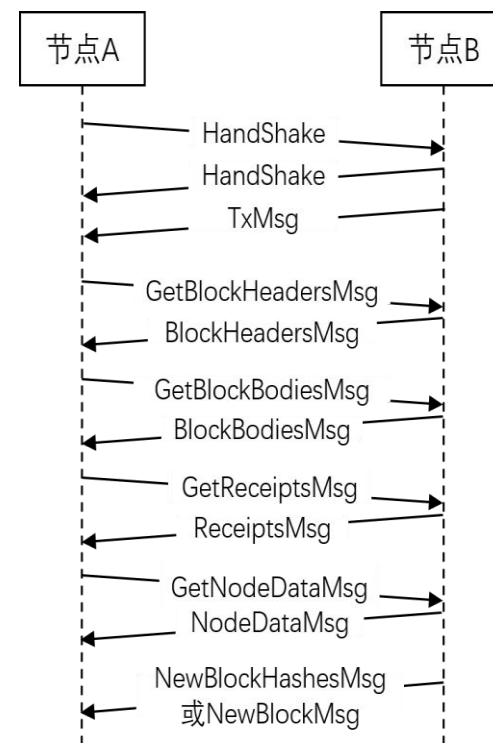
➤ 阶段二：身份认证与协议握手

- 节点A用共享密钥加密由自己使用的P2P版本号、端口号、ID等信息构成hello报文，并将hello报文向节点B发送
- 节点B使用共享密钥对hello报文进行解密，也类似地用共享密钥对自己的hello报文进行加密并发送
- 节点A完成hello报文的校验。如果双方都能接受对方的协议版本，双方开始建立通讯

区块同步

➤ 确定同步区块-同步区块头-同步区块体

- 两个节点进行简单的握手连接，连接成功后节点B将自己交易池中的交易信息同步给节点A，然后各自循环监听对方的消息
- 节点A发送GetBlockHeadersMsg获取同步区块的区块头信息，存入数据库
- 节点A依次发送GetBlockBodiesMsg、GetReceiptsMsg、GetNodeDataMsg请求获取block body、receipt（合约执行后的结果）和state（储存所有的账号状态，包括余额等信息）数据，并从回复的BlockBodiesMsg、ReceiptsMsg、NodeDataMsg报文中提取数据进行存储
- 如果节点B挖出了新的区块，会向节点A发送NewBlockMsg或者NewBlockHashesMsg进行区块同步（像这种接收到其他节点的消息，然后进行区块同步的方式被称为被动同步）
- 如果节点A收到的是NewBlockMsg，节点A会对消息中的区块直接进行验证并存入本地；如果收的是NewBlockHashesMsg，节点A会通过请求获取区块头和区块体，然后再组织成完整的区块存入本地



本章内容

4.1 P2P网络

4.2 比特币网络

4.3 以太坊网络

4.4 网络层安全

1. 分布式拒绝服务攻击

2. 延展性攻击

3. 女巫攻击

4. 路由攻击

5. 日蚀攻击

分布式拒绝服务攻击

- 将多个计算机联合起来作为攻击平台，对同一个目标发动大量的攻击请求，从而成倍地提高拒绝服务攻击的能力
- 在层叠网络（应用层）控制区块链网络中的大量在线节点，使其作为一个发起大型分布式拒绝服务攻击的放大平台
- 主动攻击是通过主动向网络中的节点发送大量的虚假索引信息，使得针对这些信息的后续访问都指向被攻击者
- 被动攻击通过修改区块链客户端或服务器软件，被动地等待来自其他节点的查询请求，再通过返回虚假响应来达到攻击效果
- 成本不高，但破坏性很强。例如：恶意矿工可以通过分布式拒绝攻击耗尽其竞争对手的网络资源，使得竞争对手被大量网络请求阻塞，从而提高自己的有效哈希率

延展性攻击

- 指在原情况不变的情况下，利用外部的虚假交易实现攻击
 - 恶意攻击者通过支付高额手续费，以高优先级进行虚假交易，使得矿工在验证这些交易的时候，发现这些交易都是虚假交易，但是它们已经在这些交易的验证上花费了相当长的时间
 - 攻击者可以侦听一笔未被确认的交易，通过修改交易签名的方式使得原有交易的交易ID发生改变，并生成一笔新的交易进行广播和确认，而参与交易的另一方无法根据原有的交易ID查询到交易的确认信息，从而可能进行重复转账并蒙受损失

女巫攻击

- 指一个攻击者节点通过向网络广播多个身份信息，非法地拥有多个身份标识，做出一些恶意行为
- 如果区块链网络中采用了投票机制，攻击者可以利用伪造的多个身份进行不公平的重复投票，从而掌握网络的控制权
- 实现反女巫攻击：采用工作量证明机制，通过验证身份的计算能力的方式，增加女巫攻击的成本；另外一种反女巫攻击的方式是身份认证

路由攻击

- 指对正常路由进行干扰从而达到攻击目标的手段
 - 区块链网络隔离成（至少）两个独立的网络，使得它们无法交换交易信息，一段时间后进行网络合并，强制性地使较短链上的所有区块被永久抛弃
 - 延迟攻击利用了区块请求在超过一定时间后才会再次发起请求的特点，通过对拦截的信息进行简单修改，延迟区块在被攻击节点的传播速度
 - 带来包括重复支付、计算能力浪费在内的潜在经济损失

日蚀攻击

- 攻击者通过侵占节点路由表的方式，控制节点的对外联系并使其保留在一个隔离的网络中，从而实施路由欺骗、拒绝服务、ID劫持等攻击行为
 - 在比特币网络中，如果攻击者节点在一个节点的路由表中占据了较高的比例，攻击者节点可以控制这个节点的正常行为，包括路由查找和资源搜索等
 - 在以太坊中，攻击者使用伪造的节点ID在受害者节点重启时重复向它发送Ping请求并占据它的K桶，使得受害者的出连接指向攻击者

本章总结

- P2P网络的特点
- P2P网络的四种拓扑形式
- 比特币、以太坊网络节点功能、类型
- 比特币、以太坊网络节点通信流程
- 比特币、以太坊网络节点地址管理、区块同步
- 比特币、以太坊网络通信加密连接机制
- 常见区块链网络攻击